

Módulo 5 – Capítulo 01 – Sección 01

Panorama general: herramientas, APIs, SDKs y frameworks para construir aplicaciones de IA

En el Módulo 4 aprendiste las arquitecturas de sistemas de IA —RAG, agentes, pipelines multi-agente, sistemas multi-modal— como patrones conceptuales que resuelven problemas de recuperación, razonamiento y acción. En este módulo aprenderás el ecosistema de herramientas con el que esas arquitecturas se implementan en producción: las APIs que proveen capacidad de inferencia, los SDKs que normalizan el acceso a esas APIs, los frameworks que reducen el código repetitivo en flujos complejos, y las herramientas de observabilidad que hacen comprensible el comportamiento de los sistemas en operación. La pregunta que guía este capítulo es: ¿qué herramienta resuelve qué problema, y cuándo añade más complejidad de la que elimina?

El ecosistema de desarrollo de IA en 2024 se articula en cuatro capas tecnológicas que se complementan sin solaparse. La capa más baja son los modelos fundacionales accesibles vía API REST: OpenAI con gpt-4o y la familia o1, Anthropic con Claude 3.5 Sonnet y Haiku, Google con Gemini 1.5 Pro y Flash vía Vertex AI o AI Studio. Estas APIs exponen el poder de inferencia de los modelos como servicios HTTP de pago por token, con autenticación por bearer token, límites de rate y contratos de SLA. Directamente sobre ellas se sitúan los SDKs oficiales —`openai` en Python y Node, `anthropic` en Python y TypeScript, `google-cloud-aiplatform`— que encapsulan la autenticación, la serialización JSON, el streaming de Server-Sent Events y la lógica de retry automático. Esta capa reduce una primera llamada al modelo a menos de diez líneas de código sin sacrificar control sobre ningún parámetro de la API.

La tercera capa son los frameworks de orquestación: LangChain, LlamaIndex y DSPy. Cada uno resuelve un problema diferente. LangChain organiza flujos de múltiples pasos con su protocolo de cadenas LCEL y ofrece una extensión para grafos de estado —LangGraph— que implementa agentes con memoria persistente, el mismo tipo de agente que el Módulo 4 describió arquitectónicamente. LlamaIndex está diseñado específicamente para el patrón RAG: su abstracción central es el índice sobre documentos, y su ecosistema de conectores de datos, query engines y pipelines de ingesta acelera la implementación de los sistemas de

recuperación-síntesis del Módulo 4. DSPy representa un paradigma diferente: en lugar de escribir prompts manualmente, define el programa como una firma de entrada/salida y deja que el optimizador encuentre los prompts más efectivos mediante evaluación automática. Finalmente, la cuarta capa son las herramientas de observabilidad —LangSmith, Langfuse, OpenTelemetry— y de evaluación —RAGAS, DeepEval, TruLens— que instrumentan el comportamiento del sistema y miden su calidad de forma sistemática.

El mapeo entre arquitecturas del Módulo 4 y herramientas del Módulo 5 es directo: RAG se implementa con LlamaIndex más una base de datos vectorial (pgvector, Qdrant, Chroma); los agentes con estado se construyen con LangGraph más tool use; los pipelines multi-agente se orquestan con LangChain o implementación directa; la observabilidad de cualquier arquitectura se instrumenta con LangSmith o Langfuse.

Componentes principales del ecosistema

- **APIs de modelos fundacionales:** endpoints HTTP REST que exponen inferencia bajo esquemas de autenticación basada en API keys, con límites de rate expresados en Tokens Per Minute (TPM) y Requests Per Minute (RPM) por tier de suscripción.
- **SDKs oficiales:** bibliotecas de cliente como `openai`, `anthropic` y `google-cloud-aiplatform` que gestionan retry automático, streaming SSE, tipado estricto de parámetros y lectura de credenciales desde variables de entorno.
- **Frameworks de orquestación:** LangChain para cadenas y agentes con estado, LlamaIndex para sistemas RAG con múltiples fuentes de datos, y DSPy para optimización declarativa de prompts mediante evaluación automática.
- **Herramientas de evaluación:** RAGAS, DeepEval y TruLens miden calidad de respuestas con métricas como faithfulness, answer relevancy y context recall de forma automatizada e integrable en pipelines de CI.
- **Herramientas de observabilidad:** LangSmith, Langfuse y OpenTelemetry con exportadores OTLP proveen trazas distribuidas, logs estructurados y métricas de latencia y costo por petición.

Nota del Arquitecto: La tentación de adoptar prematuramente todo el stack disponible —LangChain, LlamaIndex, un vector store, una plataforma de observabilidad y un framework de evaluación desde el día uno— es la fuente más común de complejidad accidental en proyectos de IA. En la práctica, el stack evoluciona por necesidad demostrada: se añade LlamaIndex cuando el volumen de

documentos supera los 10.000 chunks, se añade un framework de evaluación desde el primer sprint, y se adopta LangGraph cuando la lógica del agente requiere ciclos y estado persistente. Comenzar con el SDK puro del proveedor y un sistema de logging estructurado ya es un stack de producción viable para muchos casos de uso.

La maestría del AI Engineer no reside en dominar todas las herramientas del ecosistema sino en elegir con precisión qué capa usar para cada problema: las APIs para el acceso al modelo, los SDKs para el código de integración, los frameworks solo cuando la complejidad del flujo lo justifica, y la observabilidad siempre. Las secciones siguientes de este capítulo profundizan en cada una de estas capas: las diferencias entre las APIs de los tres proveedores principales, la instalación y configuración de los SDKs oficiales, la gestión segura de credenciales, y los criterios técnicos y económicos para elegir entre las herramientas disponibles según el caso de uso.

Principio rector: El ecosistema de IA es una pila de abstracciones donde cada capa tiene un costo en complejidad, dependencias y curva de aprendizaje; la maestría del AI Engineer reside en elegir el nivel mínimo necesario para resolver el problema sin introducir dependencias que el sistema no necesita todavía.

Módulo 5 – Capítulo 01 – Sección 02

APIs de modelos fundacionales: OpenAI, Anthropic, Google — estructura y autenticación

¿Cuán diferentes son realmente las APIs de los tres proveedores dominantes del mercado? La respuesta importa porque determina cuánto esfuerzo cuesta construir una abstracción portable y qué capacidades diferenciadoras pierde el sistema si se abstrae demasiado. Las APIs de OpenAI, Anthropic y Google Vertex AI comparten el modelo HTTP REST con autenticación por token y serialización JSON, pero difieren en la estructura del mensaje, los mecanismos de autenticación, la forma de expresar el contexto del sistema y las capacidades multimodales. Comprender estas diferencias en detalle —no en abstracto— es la diferencia entre una integración que funciona correctamente la primera vez y una que falla de formas sorprendentes a las tres semanas de estar en producción.

La API de OpenAI es la más establecida y la más imitada: su endpoint `https://api.openai.com/v1/chat/completions` recibe un cuerpo JSON con el campo `model` (ej. `gpt-4o-2024-08-06`, siempre con la fecha para evitar actualizaciones silenciosas), un array `messages` con objetos `{role, content}` donde el rol `system` es el primer elemento del array, y parámetros de inferencia como `temperature`, `max_tokens`, `top_p` y `stream`. La autenticación se realiza con el header `Authorization: Bearer $OPENAI_API_KEY`. Los tiers de rate limit escalan desde Tier 1 con 500 RPM y 30.000 TPM hasta Tier 5 con 10.000 RPM y 30.000.000 TPM, determinados por el gasto histórico acumulado. Los errores 429 incluyen el header `Retry-After` que indica los segundos de espera antes del siguiente intento.

La API de Anthropic tiene una diferencia estructural importante: el campo `system` no es parte del array `messages` sino un campo de primer nivel en el cuerpo del request. Esta separación no es cosmética —tiene implicaciones en el prompt caching, porque el campo `system` puede marcarse con `cache_control: {"type": "ephemeral"}` independientemente de los mensajes del turno, lo que permite cachear instrucciones largas del sistema mientras los mensajes del usuario varían libremente. La autenticación requiere dos headers en cada request: `x-api-key: $ANTHROPIC_API_KEY` y `anthropic-version: 2023-06-01`, siendo el segundo obligatorio para garantizar la compatibilidad con la versión del protocolo que el

código espera. El código de error 529 es exclusivo de Anthropic e indica sobrecarga temporal del servicio, distinto del 429 de rate limit: ambos son retryables pero con estrategias diferentes.

Google ofrece dos rutas de acceso a Gemini. Vertex AI es la ruta enterprise: autenticación con OAuth2 via service account JSON o Application Default Credentials (ADC), endpoint dinámico por región `{region}-aiplatform.googleapis.com`, soporte nativo multimodal con `inline_data` y `file_data` que permiten intercalar texto, imagen, audio y video en el mismo mensaje, y acceso a controles de seguridad VPC, CMEK y VPC Service Controls. AI Studio es la ruta de desarrollo rápido: autenticación simplificada con `x-goog-api-key`, misma capacidad de modelo sin los controles enterprise, sin SLA de producción. La elección entre Vertex AI y AI Studio es siempre la misma: Vertex AI en producción con datos sensibles, AI Studio en prototipado y desarrollo.

Aspectos técnicos de las APIs principales

- **OpenAI API:** endpoint `/v1/chat/completions`, header `Authorization: Bearer`, campo `system` dentro del array `messages`, tiers de rate limit por gasto acumulado, soporte de `response_format: {"type": "json_schema"}` para structured outputs garantizados.
- **Anthropic API:** campo `system` de primer nivel (no en `messages`), headers `x-api-key` y `anthropic-version` obligatorios, error 529 para sobrecarga temporal, `cache_control` para prompt caching granular por bloque de contenido.
- **Google Vertex AI:** autenticación OAuth2 con service accounts, endpoint regional dinámico, soporte multimodal nativo con `contents[].parts[]`, controles enterprise de VPC y CMEK, `generationConfig.responseMimeType: "application/json"` para salida estructurada.
- **Google AI Studio:** `x-goog-api-key` para desarrollo, misma capacidad del modelo sin controles enterprise, adecuado para prototipado y evaluación de modelos.
- **Códigos de error críticos:** 400 (parámetros inválidos o contexto excedido), 401 (API key ausente o revocada), 422 (contenido que viola políticas de uso), 429 (rate limit con `Retry-After`), 529 (sobrecarga temporal de Anthropic, retryable con backoff).

Nota del Arquitecto: El error más frecuente al integrar múltiples proveedores es asumir que el campo `system` funciona igual en todos ellos. En Anthropic, mover el contenido de contexto estático (reglas de negocio, ejemplos few-shot, documentos de referencia) al campo `system` separado habilita el prompt caching sin cambiar el

resto de la llamada. En OpenAI, el mismo contenido va como primer mensaje del array con `role: "system"` y el caching es automático para prefijos de más de 1.024 tokens. Entender esta diferencia antes de diseñar el sistema de caching puede ahorrar semanas de refactorización posterior.

Las diferencias estructurales entre APIs son argumentos concretos a favor de encapsular el acceso al proveedor en un adaptador interno: cuando el código de la aplicación llama a `llm.complete(system_prompt, messages)` en lugar de llamar directamente al SDK de cada proveedor, cambiar el proveedor o aprovechar una nueva capacidad de autenticación requiere modificar solo el adaptador. La siguiente sección explora precisamente cómo los SDKs oficiales de cada proveedor encapsulan estas diferencias y simplifican las primeras llamadas al modelo.

Para recordar: Cada proveedor impone su propio contrato de autenticación, estructura de mensajes y semántica del campo `system`; abstraer estas diferencias en un adaptador interno es más robusto que depender de wrappers de terceros para aplicaciones de producción donde los detalles del protocolo importan.

Módulo 5 – Capítulo 01 – Sección 03

SDKs oficiales: instalación, configuración y primeras llamadas

La diferencia entre llamar a una API de LLM con `urllib.request` o `httpx` manualmente y usar el SDK oficial del proveedor no es solo de comodidad sintáctica: los SDKs encapsulan decisiones de ingeniería que tardarían días en implementarse correctamente desde cero. El retry automático con backoff exponencial ante errores transitorios, el streaming de Server-Sent Events con buffering correcto, el tipado estricto que detecta parámetros inválidos antes de hacer la llamada, la rotación automática de bases de URL para diferentes regiones, y la gestión del ciclo de vida de las conexiones HTTP con pooling —todo esto viene incluido y probado. La pregunta no es si usar el SDK sino cuál, cuándo actualizarlo y qué aspectos de su configuración merecen atención al momento de la instalación.

El SDK de OpenAI se instala con `pip install openai` y su instancia principal es `client = OpenAI(api_key=os.environ["OPENAI_API_KEY"])`. Si la variable de entorno `OPENAI_API_KEY` existe en el entorno del proceso, el SDK la lee automáticamente sin que el argumento `api_key` sea necesario. La llamada síncrona básica es `response = client.chat.completions.create(model="gpt-4o-2024-08-06", messages=[{"role": "user", "content": "Hola"}], max_tokens=256)` y la respuesta se accede como `response.choices[0].message.content`. La variante asíncrona usa `AsyncOpenAI` y es directamente compatible con frameworks como FastAPI. El SDK también expone el atributo `response.usage` con `prompt_tokens`, `completion_tokens` y `total_tokens`, indispensable para el seguimiento de costos.

El SDK de Anthropic (`pip install anthropic`) sigue una arquitectura similar: `client = anthropic.Anthropic()` lee `ANTHROPIC_API_KEY` del entorno, y `client.messages.create(model="claude-3-5-sonnet-20241022", max_tokens=256, system="Eres un asistente técnico.", messages=[{"role": "user", "content": "Hola"}])` realiza la llamada. Una diferencia notable en el tipado es que el SDK de Anthropic genera sus tipos desde el schema OpenAPI de la API, lo que significa que los atributos de la respuesta son objetos fuertemente tipados — `response.content[0].text`,

`response.usage.input_tokens` — en lugar de dicts. El SDK también expone `client.messages.count_tokens()` para contar tokens de un mensaje antes de enviarlo, útil para decisiones de costo previas a la llamada.

Google ofrece dos SDKs según la ruta de acceso. Para AI Studio: `pip install google-generativeai` y `genai.configure(api_key=os.environ["GOOGLE_API_KEY"])`; la llamada es `model = genai.GenerativeModel("gemini-1.5-pro"); response = model.generate_content("Hola")`. Para Vertex AI: `pip install google-cloud-aiplatform` con autenticación por Application Default Credentials (`gcloud auth application-default login` en local, service account en CI/CD); la llamada requiere configurar primero el proyecto y la región con `vertexai.init(project=PROJECT_ID, location="us-central1")` antes de cualquier llamada al modelo.

Componentes principales de los SDKs

- **Gestión de autenticación:** los SDKs leen la API key desde variable de entorno por convención (`OPENAI_API_KEY`, `ANTHROPIC_API_KEY`) sin requerir configuración explícita, evitando el hardcoding de credenciales en el código fuente.
- **Tipado estricto con autocompletado:** los SDKs modernos usan tipos generados desde especificaciones OpenAPI, proporcionando autocompletado en IDEs y detectando parámetros inválidos antes de la llamada, reduciendo el tiempo de depuración.
- **Retry automático con backoff:** el SDK de OpenAI reintenta hasta 2 veces por defecto con jitter ante errores transitorios; el SDK de Anthropic tiene `max_retries` configurables (default 2) para errores 429 y 529.
- **Streaming nativo:** `stream=True` en OpenAI y el context manager `with client.messages.stream(...) as stream` en Anthropic devuelven iteradores de eventos delta, con acumulación automática de la respuesta completa disponible via `stream.get_final_message()`.
- **Configuración de timeout y base_url:** ambos SDKs permiten configurar `timeout` (default 600s en OpenAI) y `base_url` para redirigir a proxies, gateways internos o servicios compatibles como Azure OpenAI Service con su propio `api_version`.

Nota del Arquitecto: El parámetro `base_url` de los SDKs de OpenAI y Anthropic es más valioso de lo que parece inicialmente. En sistemas con múltiples proveedores, configurar `base_url` para apuntar a un proxy interno como LiteLLM permite redirigir todas las llamadas a través de un gateway centralizado sin cambiar

ninguna línea de código de aplicación: el gateway centraliza el logging, el rate limiting interno y el failover entre proveedores. Configurar el gateway desde el inicio del proyecto, incluso si al principio solo hay un proveedor, es una decisión de diseño de bajo costo con alto retorno operacional.

El patrón de uso recomendado para proyectos de producción es instanciar el cliente una sola vez al arrancar la aplicación y reutilizarlo en todas las llamadas, en lugar de instanciarlo por request. El cliente mantiene internamente un pool de conexiones HTTP que se reutilizan entre llamadas, reduciendo la latencia de establecimiento de conexión. En aplicaciones web con FastAPI o Flask, el cliente se instancia como dependencia de la aplicación y se inyecta vía `Depends()` o como variable de módulo. La siguiente sección aborda el aspecto de seguridad más importante de trabajar con SDKs: la gestión de las credenciales que esos clientes necesitan para autenticarse.

Principio rector: Usar el SDK oficial en lugar de llamadas HTTP directas no solo reduce el código necesario; garantiza compatibilidad automática con actualizaciones de protocolo del proveedor, retry correcto ante errores transitorios, y tipado estricto que previene errores silenciosos en producción.

Módulo 5 – Capítulo 01 – Sección 04

Gestión de credenciales, variables de entorno y seguridad básica

Las API keys de modelos fundacionales no son credenciales ordinarias: conceden acceso irrestricto a la cuota de gasto del proyecto y, en muchos casos, a los datos que se envían como contexto al modelo. Una key de OpenAI filtrada en un repositorio público puede generar cargos de miles de dólares en horas antes de ser detectada y revocada —incidentes de este tipo están documentados en GitHub con frecuencia creciente desde 2023. Una key de Anthropic comprometida puede también exponer los prompts y documentos que el sistema envía como contexto, que pueden contener información confidencial de negocio o datos personales de usuarios. La gestión segura de credenciales no es un aspecto avanzado de la operación: es un requisito desde el primer commit del repositorio, antes de que la primera key sea generada.

La práctica universal para desarrollo local es almacenar las API keys en un archivo `.env` en la raíz del proyecto, cargado con `python-dotenv` mediante `from dotenv import load_dotenv; load_dotenv()` al inicio del proceso. Este archivo contiene las keys en texto plano — `OPENAI_API_KEY=sk-...` — y debe agregarse al `.gitignore` en el mismo commit en que se crea el repositorio, antes de que ninguna key exista. La secuencia correcta es: crear el repositorio, agregar `.env` y `.env.local` a `.gitignore`, crear el archivo `.env.example` con las variables de entorno esperadas pero sin valores reales (para que el equipo sepa qué keys configurar), y solo entonces generar las API keys y agregarlas al `.env` local. El orden importa: decenas de incidentes de seguridad documentados ocurrieron porque el desarrollador creó el `.gitignore` después de hacer el primer commit con el `.env` incluido.

En entornos de producción, las variables de entorno se inyectan desde un gestor de secretos externo en lugar de archivos locales. AWS Secrets Manager permite almacenar los secretos como pares clave-valor con rotación automática y acceso controlado por IAM roles; las aplicaciones en ECS o Lambda los leen via SDK de AWS sin que el valor aparezca en logs ni en el filesystem del contenedor. HashiCorp Vault ofrece rotación dinámica de secretos con leases temporales que expiran; es la opción dominante en infraestructuras multi-cloud. Google Secret Manager y Azure Key Vault son las equivalentes en sus respectivos clouds, con

integración nativa con los servicios de cómputo del proveedor. Los pipelines de CI/CD — GitHub Actions, GitLab CI, CircleCI— tienen mecanismos propios de secretos que inyectan las variables en el entorno de ejecución sin exponerlas en los logs.

La detección preventiva es el segundo pilar de la gestión segura de credenciales. Herramientas como `detect-secrets`, `truffleHog` y `gitleaks` se instalan como hooks de pre-commit: antes de que cualquier commit llegue al repositorio, el hook escanea los archivos por patrones de API keys (el SDK de OpenAI usa el prefijo `sk-`, las keys de Anthropic usan `sk-ant-`) y rechaza el commit si detecta alguno. Configurar estos hooks en el repositorio como parte del archivo `.pre-commit-config.yaml` garantiza que todos los miembros del equipo los ejecuten sin necesidad de recordarles.

Puntos críticos de seguridad en credenciales de IA

- **Regla de oro:** nunca hardcodear API keys en el código fuente; usar siempre `os.environ.get("API_KEY")` o `os.getenv("API_KEY")` como única forma de acceso dentro del código de la aplicación, con fallo explícito si la variable no existe.
- **Archivo `.env` local:** usar `python-dotenv` para cargar variables en desarrollo, con `.env`, `.env.local` y `.env.*.local` siempre presentes en `.gitignore` desde el primer commit del repositorio.
- **Gestores de secretos en producción:** AWS Secrets Manager, Azure Key Vault o Google Secret Manager proveen rotación automática, auditoría de accesos y acceso controlado por IAM roles sin hardcodear credenciales en el runtime de la aplicación.
- **Pre-commit hooks de detección:** configurar `detect-secrets` o `gitleaks` como hooks de pre-commit para detectar patrones de API keys antes de que lleguen al repositorio remoto.
- **Keys separadas por ambiente:** OpenAI y Anthropic permiten crear múltiples API keys con nombres descriptivos; usar keys diferentes para desarrollo, staging y producción con límites de gasto diferenciados por ambiente previene que experimentos en desarrollo consuman cuota de producción.

Nota del Arquitecto: En organizaciones con múltiples equipos usando el mismo proveedor, crear proyectos separados por equipo o feature en la consola del proveedor (OpenAI Projects, Anthropic Workspaces) permite aislar el gasto y los límites de rate entre equipos sin gestionar una única key compartida. Una key compartida convierte cada incidente de rate limiting en un problema que afecta a

todos los equipos simultáneamente; keys independientes por equipo aíslan el impacto y facilitan la atribución de costos en las revisiones presupuestarias.

La gestión segura de credenciales establece el primer perímetro de seguridad del sistema de IA. El segundo perímetro —validar la entrada del usuario para prevenir prompt injection y sanitizar la salida antes de exponerla— se aborda en el Capítulo 10. Lo que la sección siguiente explora es un problema diferente al de la seguridad: la elección de qué API, SDK y modelo usar según los requisitos técnicos y económicos del caso de uso, que es la decisión de diseño con mayor impacto en el costo y la calidad del sistema a largo plazo.

Idea central: Una credencial de API de modelo fundacional comprometida no solo expone gasto monetario sino potencialmente datos sensibles del contexto enviado al modelo; la gestión segura de secretos es un requisito de producción desde el primer día, no una consideración posterior de hardening.

Módulo 5 – Capítulo 01 – Sección 05

Elección de herramientas según el caso de uso: criterios técnicos y económicos

La elección de qué API, SDK, modelo y framework usar no debe tomarse por popularidad ni por la última tendencia en Hacker News sino por una evaluación rigurosa y empírica de los requisitos específicos del caso de uso. Este punto es más importante de lo que parece: la elección de modelo incorrecto puede multiplicar el costo por 20x sin mejora medible en calidad, o introducir latencia inaceptable en una interfaz conversacional que requiere respuesta en menos de un segundo. El proceso correcto siempre comienza con la medición: definir el conjunto de criterios relevantes para el caso de uso, medir el desempeño de los candidatos sobre un dataset representativo del dominio, y tomar la decisión con evidencia empírica, no con supuestos.

La latencia de Time-to-First-Token (TTFT) es el criterio dominante en interfaces conversacionales en tiempo real. Un chatbot de atención al cliente donde el usuario espera una respuesta en una interfaz de chat tiene un umbral de experiencia de usuario aceptable de 500ms a 1 segundo de TTFT; si el sistema supera esos valores regularmente, los usuarios perciben el sistema como lento aunque la respuesta sea excelente. Modelos como Claude 3.5 Haiku y gpt-4o-mini tienen TTFT típicos de 200-400ms; modelos más grandes como Claude 3.5 Sonnet y gpt-4o alcanzan 400-800ms bajo carga normal; modelos de razonamiento como o1 pueden tomar de 5 a 60 segundos antes del primer token porque ejecutan razonamiento interno extenso. La elección del modelo de razonamiento se justifica solo en tareas donde la calidad del razonamiento es crítica y la latencia no importa —análisis de código complejo, evaluación de decisiones estratégicas— nunca en interfaces de chat.

El costo por tarea es el segundo criterio. El cálculo correcto es $(input_tokens \times precio_entrada + output_tokens \times precio_salida) \times volumen_diario \times 30$, y los números resultan reveladores. Un sistema de clasificación de intención de usuario con 5 categorías que procesa 100.000 requests diarios con 500 tokens de entrada y 50 tokens de salida cuesta con Claude 3.5 Haiku ($\$0.80/M$ entrada, $\$4/M$ salida) aproximadamente $(500 \times 0.80 + 50 \times 4.0) / 1M \times 100.000 \times 30 = \$720/mes$; el mismo sistema con Claude 3.5 Sonnet ($\$3/M$

`entrada, $15/M salida`) costaría `$3.750/mes` —5x más caro— sin ninguna mejora en la tarea de clasificación que un modelo pequeño resuelve con 95%+ de accuracy. Ejecutar el benchmark antes de comprometer la arquitectura con un modelo es la inversión de tiempo con mayor retorno en proyectos de IA de volumen alto.

Los requisitos de privacidad y soberanía de datos son el tercer eje. Las APIs de los tres proveedores principales envían los datos del prompt a servidores externos; para datos regulados (HIPAA, GDPR con datos especialmente sensibles, datos de defensa o inteligencia) esto puede ser un bloqueador. Las alternativas son: Azure OpenAI Service con contrato de datos dentro del tenant del cliente, Vertex AI con VPC Service Controls que mantienen los datos dentro de la VPC del cliente, o modelos self-hosted con Ollama (para desarrollo y bajo volumen) o vLLM (para producción con GPU dedicada). Ollama permite ejecutar modelos como Llama 3.1 70B, Mistral 7B o Qwen 2.5 72B localmente con una API compatible con OpenAI vía `base_url="http://localhost:11434/v1"`, sin costo de inferencia pero con costo de hardware. vLLM es el servidor de inferencia de alto throughput para GPU, con soporte de batching continuo, tensor parallelism y compatibilidad con la API de OpenAI que permite usar el SDK de OpenAI sin modificación.

Criterios técnicos y económicos de selección

- **Latencia vs capacidad:** Claude 3.5 Haiku con TTFT típico <500ms para interfaces conversacionales; gpt-4o para tareas que requieren razonamiento amplio con latencia moderada; o1 o Claude 3.5 Opus para razonamiento profundo donde la latencia no es crítica.
- **Costo por tarea medido:** calcular `(input_tokens × precio_entrada + output_tokens × precio_salida) × volumen`; para clasificación y extracción simple, los modelos pequeños ofrecen 90-97% de la calidad del modelo grande a 5-20x menor costo.
- **Ventana de contexto:** Gemini 1.5 Pro con 1 millón de tokens para análisis de codebase completa; Claude 3.5 Sonnet con 200K para documentos largos; gpt-4o con 128K para conversaciones largas; la elección depende del tamaño del corpus que debe caber en una sola llamada.
- **Privacidad y cumplimiento:** APIs de terceros para datos no regulados; Azure OpenAI o Vertex AI con VPC para datos regulados dentro del cloud del proveedor; vLLM o Ollama para datos que no pueden salir de la infraestructura propia.
- **Ecosistema y estabilidad operacional:** proveedores con SLA documentado, changelog público, foro activo de soporte y latencia de resolución de incidentes de <24h

tienen ventaja en proyectos de producción donde los incidentes deben resolverse rápidamente.

Nota del Arquitecto: El criterio que más frecuentemente se omite en las evaluaciones de modelo es la estabilidad del comportamiento ante cambios del proveedor. OpenAI actualiza los modelos bajo el mismo identificador sin aviso previo; Anthropic publica notas de deprecación con 6 meses de antelación. Para sistemas donde la consistencia de comportamiento es crítica (clasificadores con thresholds calibrados, sistemas donde los usuarios reportan inconsistencias), anclar el identificador de modelo con fecha (`claude-3-5-sonnet-20241022` en lugar de `claude-3-5-sonnet`) y monitorear el campo `model` devuelto en cada respuesta detecta actualizaciones silenciosas del proveedor antes de que afecten al sistema.

La elección de herramientas no es definitiva: se revisita cuando el volumen escala más allá del tier de rate limit actual, cuando aparece un nuevo modelo con mejor relación costo-calidad, o cuando los requisitos de privacidad del cliente cambian. La siguiente sección cierra el capítulo con una síntesis del stack mínimo viable: el conjunto acotado de herramientas con el que un AI Engineer puede construir y operar un sistema de IA en producción sin añadir complejidad innecesaria.

Buena práctica: Evaluar modelos con un benchmark representativo del caso de uso real — al menos 100-200 ejemplos del dominio específico con métricas de calidad y costo medidas simultáneamente— antes de comprometer la arquitectura con un proveedor o modelo específico.

Módulo 5 – Capítulo 01 – Sección 06

Cierre: el stack mínimo viable para un AI Engineer

El síndrome del stack sobreingeniado es una patología recurrente en equipos que comienzan a construir con IA: ante la abundancia de herramientas disponibles, el equipo adopta LangChain para orquestación, LlamaIndex para recuperación, pgvector para almacenamiento vectorial, Langfuse para observabilidad, RAGAS para evaluación y Redis para caché semántica —todo antes de tener un solo usuario real. El problema no es que alguna de estas herramientas sea mala; es que cada una introduce dependencias, una curva de aprendizaje, versiones que actualizar y un punto potencial de fallo adicional. El stack sobreingeniado no falla por ser complejo sino por ser complejo demasiado pronto, antes de que la evidencia de producción justifique cada capa de abstracción.

El stack mínimo viable para construir y operar un sistema de IA en producción converge en un conjunto acotado de herramientas que resuelven los problemas fundamentales sin sobrecarga prematura. La base es el SDK oficial del proveedor: `openai`, `anthropic` o `google-generativeai`, con retry automático, tipado estricto y streaming incluidos. Sobre esta base se añade gestión segura de credenciales — `.env` más `python-dotenv` en desarrollo, un gestor de secretos en producción— y logging estructurado con Python `structlog` o el módulo estándar `logging` configurado en formato JSON, capturando como mínimo el modelo usado, los tokens de entrada y salida, la latencia, el costo estimado y el `request_id`. Este núcleo de tres elementos es suficiente para operar un sistema de IA en producción y diagnosticarlo cuando algo falla.

La primera extensión que se justifica casi siempre en el primer sprint, no como añadido tardío, es una herramienta de observabilidad: Langfuse (open source y self-hostable con Docker Compose en menos de una hora) o LangSmith si el proyecto ya usa LangChain. Esta herramienta captura trazas de cada llamada al LLM con su prompt completo, la respuesta, los tokens, la latencia y el costo, creando el primer activo de observabilidad del sistema. Sin esta capa, el equipo opera ciegamente: no puede responder "¿cuánto costó el sistema ayer?", "¿qué porcentaje de llamadas están fallando?" ni "¿qué prompts están generando las respuestas de menor calidad?".

Las extensiones adicionales se añaden cuando la evidencia las justifica. Un framework de orquestación como LangChain o LlamaIndex se incorpora cuando el flujo supera los 3-4 pasos interdependientes y la implementación directa requiere más de 200 líneas de código para manejar correctamente todos los casos borde. Una base de datos vectorial se agrega cuando el volumen de documentos supera los 10.000 chunks o cuando la búsqueda semántica en memoria es demasiado lenta. Un sistema de evaluación automatizado con RAGAS o DeepEval se configura desde el primer sprint, pero sobre un dataset pequeño de 20-50 casos representativos, no como un sistema elaborado. La secuencia es siempre: operabilidad primero, luego escala, luego sofisticación.

Componentes del stack mínimo viable

- **SDK del proveedor:** `openai`, `anthropic` o `google-generativeai` para las llamadas al modelo con tipado estricto, retry automático y soporte de streaming incorporado.
- **Gestión de secretos:** `.env` más `python-dotenv` en desarrollo local; Secret Manager o variables de entorno del CI/CD en producción; nunca hardcodear API keys en el código fuente.
- **Logging estructurado:** captura de modelo, tokens de entrada, tokens de salida, latencia, costo estimado, `request_id` y errores en formato JSON, almacenado en la plataforma de logging del cloud o en archivos locales.
- **Observabilidad básica:** Langfuse self-hosted o LangSmith para trazas de llamadas, costos acumulados, y comparación de versiones de prompts; se configura en el mismo sprint que el primer endpoint de IA.
- **Testing desde el primer día:** pytest con mocks del cliente LLM para unit tests de la lógica Python, y un dataset pequeño de casos representativos para regression testing; no un sistema elaborado, pero sí la infraestructura básica desde el inicio.

El conjunto de herramientas descrito en este capítulo —APIs, SDKs, gestión de credenciales, observabilidad básica y testing— es el sustrato sobre el que se construyen todos los sistemas del módulo. Los capítulos siguientes profundizan en cada aspecto: el Capítulo 2 disecciona el protocolo de comunicación con los modelos en detalle, el Capítulo 3 explora los frameworks de orquestación con sus criterios de adopción y sus limitaciones, y los capítulos posteriores abordan el testing, el CI/CD, la evaluación de calidad y la optimización de costos como disciplinas de ingeniería de producción. La progresión es deliberada: cada capa del stack se introduce cuando el lector tiene el contexto para entender por qué existe y qué problema específico resuelve.

"Simplicity is a prerequisite for reliability." — Edsger Dijkstra. En AI Engineering, cada herramienta adicional en el stack introduce un punto potencial de fallo, una versión que actualizar y un concepto que el equipo debe dominar; el stack mínimo viable no es una limitación de capacidades sino la base de un sistema comprensible, diagnosticable y confiable a lo largo del tiempo.